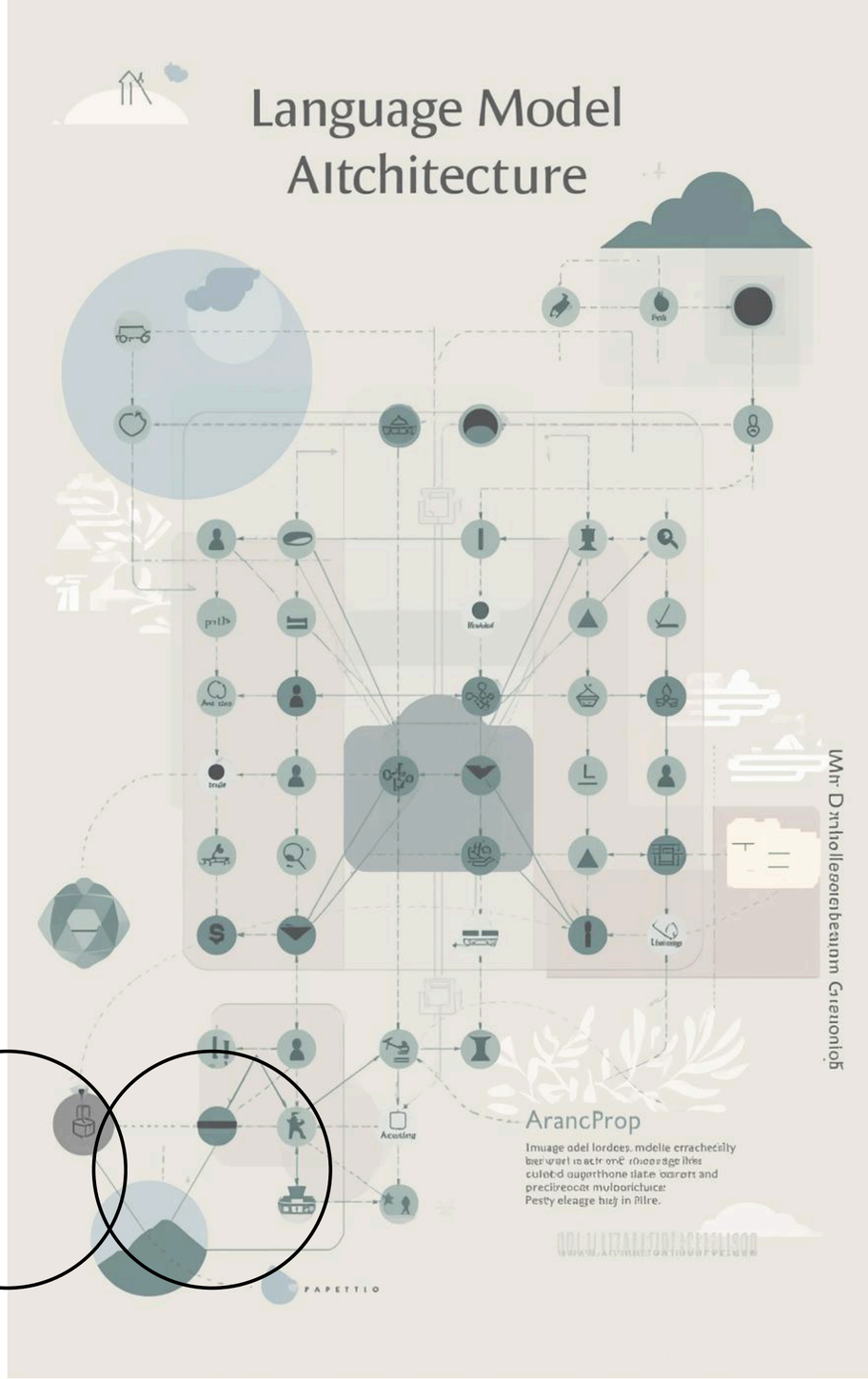
Retrieval-Augmented Language Modeling



Presented by Mert KARAHAN

Motivation for RALM

Overview of LLMs and their key limitations

Rise of LLMs

Large Language Models have transformed NLP, showcasing remarkable capabilities in understanding and generating human-like text across various domains.

Limitations of LLMs

Despite their strengths, LLMs lack access to current external knowledge, leading to potential factual inaccuracies and unverified information in responses.

Need for RALM

Retrieval-Augmented Language Modeling addresses LLM limitations by integrating external documents, enhancing reliability and adaptability in specialized and private domains.

Challenges with Prior RALM

Understanding Existing Limitations

Architectural Changes

Many RALM systems require significant **architectural modifications** to the underlying language model, complicating implementation and potentially introducing new challenges during deployment and maintenance phases.

Additional Training

Most prior RALM approaches necessitate **extra pretraining or fine-tuning**, which not only increases the resource consumption but also extends the timeline for effective deployment in real-world applications.

Complex Deployment

The intricate deployment requirements of earlier RALM methods hinder their adoption, particularly in **API-only environments**, making it challenging for developers to integrate these systems efficiently into existing pipelines.

Proposed In-Context RALM

Simple Integration

The approach seamlessly integrates retrieved documents directly into input without altering the language model's architecture or training.

Enhanced Efficiency

It utilizes off-the-shelf models, maximizing their capabilities without the need for complex setup or ongoing training requirements.

Compatibility

Fully compatible with API-based language models, this method ensures accessible deployment across various applications and systems.

Performance Scaling

The performance benefits are significant, with results comparable to much larger models while simplifying the deployment process considerably.

In-Context RALM Benefits

Advantages of a New Approach

Performance Enhancement

The In-Context RALM approach significantly boosts performance by achieving results comparable to increasing model size by 2–3×. This technique allows smaller models to leverage external documents effectively, enhancing their output quality and accuracy.

Simplified Deployment

This method enables easy integration without the need for complex architectural changes or retraining processes. It ensures that users can implement RALM solutions seamlessly within existing frameworks, promoting widespread adoption in various applications.

Experimental Setup

Objective

To evaluate the effectiveness, generality, and scalability of In-Context RALM across multiple corpora, model families, and retrieval configurations.

Evaluation Procedure

Models generate text with and without retrieval augmentation.
Retrieval is applied at fixed strides, and perplexity is measured on held-out datasets to quantify performance improvements.

Retrieval Corpus Construction

Wikipedia (Dec 2018) and dataset-specific training sets are divided into non-overlapping passages of ~100 words. Retrieved passages are truncated to 256 tokens to fit model context limits.

Implementation Details

All experiments use a max sequence length of 1,024 tokens for comparability.
Sparse retrieval uses Pyserini (BM25), while dense retrieval uses FAISS-based exact search.
All LM experiments use HuggingFace Transformers.

Study Scope

Datasets

The study evaluates In-Context RALM across five diverse language modeling datasets — WikiText-103, ArXiv, StackExchange, FreeLaw, and RealNews — along with two open-domain QA datasets, Natural Questions and TriviaQA. This ensures broad domain coverage and tests the robustness of retrieval augmentation.

Model Families

Experiments span four major LM families: GPT-2 (110M–1.5B), GPT-Neo/J (1.3B–6B), OPT (125M–66B), and LLaMA (7B–33B). This wide range allows examination of retrieval effectiveness across model sizes and architectures.

Retrieval Methods

Both sparse and dense retrieval approaches are evaluated. BM25 serves as the primary sparse retriever, while BERT-base, Contriever, and Spider represent dense neural alternatives. Additional reranking experiments employ RoBERTa-based models.

Retrieval Settings

The study systematically explores retrieval stride, query length, and document reranking. These controlled variations identify optimal configurations for maximizing the performance of In-Context RALM.

In-Context RALM Delivers Large Gains with Simple Off-the-Shelf Retrieval

- In-Context RALM with BM25 (ℓ = 32, s = 4) yields substantial perplexity reductions.
- Matches performance of models 2–3× larger across all GPT-2 sizes.
- Scales to large models: OPT-6.7B → OPT-66B equivalent.
- Generalizes across all corpora and domains.
- Strong example: GPT-2 345M + BM25 > GPT-2 762M,
- and with reranking → beats GPT-2 1.5B.

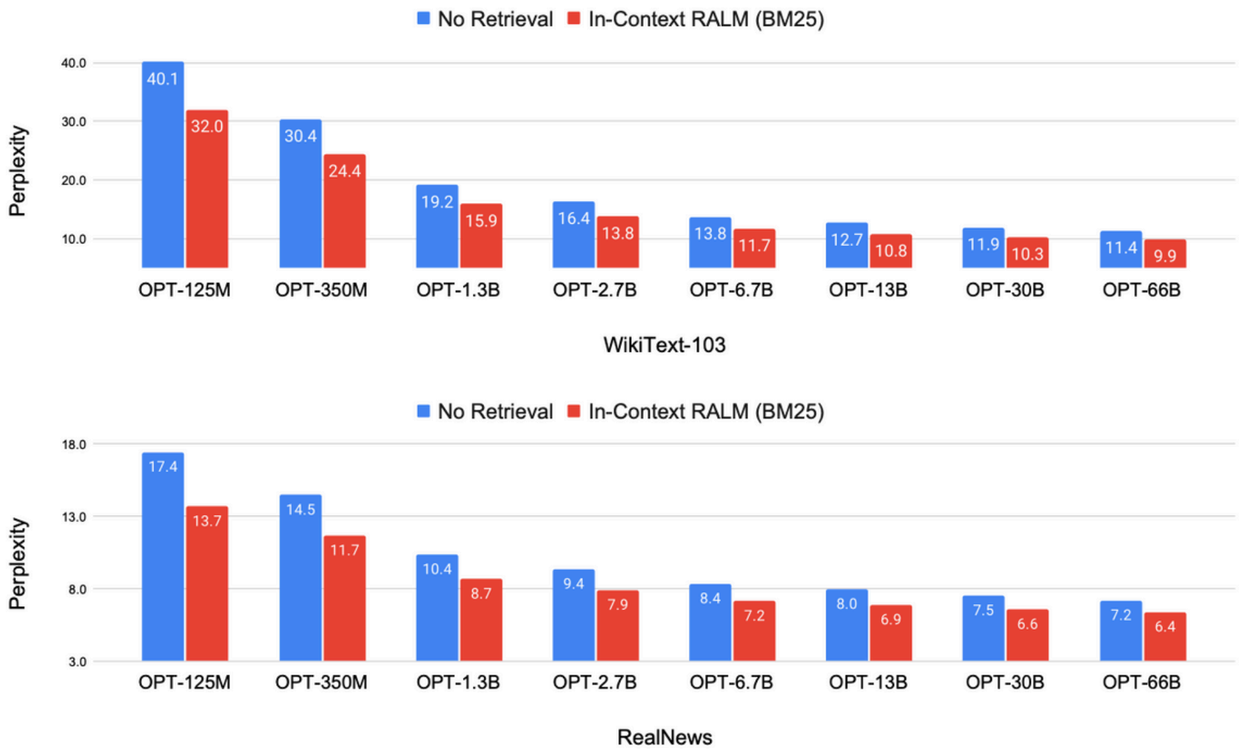


Figure 4: Results of OPT models (Zhang et al., 2022) on the test set of WikiText-103 (word-level perplexity) and the development set of RealNews (token-level perplexity). In-Context RALM models use a BM25 retriever with $s = 4$ (*i.e.*, the retriever is called every four tokens) and $\ell = 32$ (*i.e.*, the retriever query is comprised of the last 32 tokens of the prefix). *In-Context RALM with an off-the-shelf retriever improved the performance of a 6.7B parameter OPT model to match that of a 66B parameter OPT model.*

1. *Retrieval-augmented models (red) consistently achieve perplexity comparable to much larger baselines (blue)*

Retrieval Frequency Improves LM Performance

- Frequent retrieval (stride s = 4) yields the strongest perplexity improvements.
- Larger strides (e.g., s = 16, 32, 64) steadily degrade performance.
- Supports the idea that retrieval should rely on recent prefix tokens for relevance.
- Demonstrates the importance of maintaining a high-resolution grounding signal.
- RETRO's large stride (s = 64) explains its reduced effectiveness.

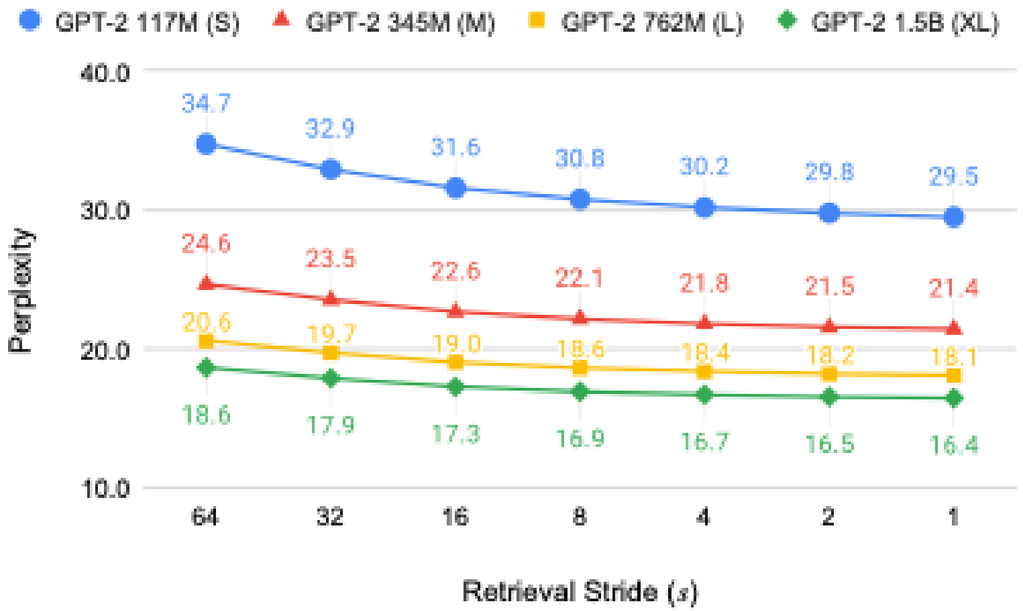


Figure 5: An analysis of perplexity as a function of s, the *retrieval stride*, *i.e.*, the number of tokens between consecutive retrieval operations, on the development set of WikiText-103. Throughout the paper, we use $s = 4$ to balance perplexity and runtime.

Perplexity improves consistently as retrieval becomes more frequent.

Query Length Tradeoff

- An optimal query length of ~32 tokens significantly improves retrieval relevance.
- Short queries lack context, leading to weak retrieval grounding.
- Long queries dilute recency, lowering the effectiveness of In-Context RALM.
- BM25 performs best at $\ell = 32$, while dense retrievers peak at $\ell = 64$ but still underperform.
- Balances local context with global semantic signal.

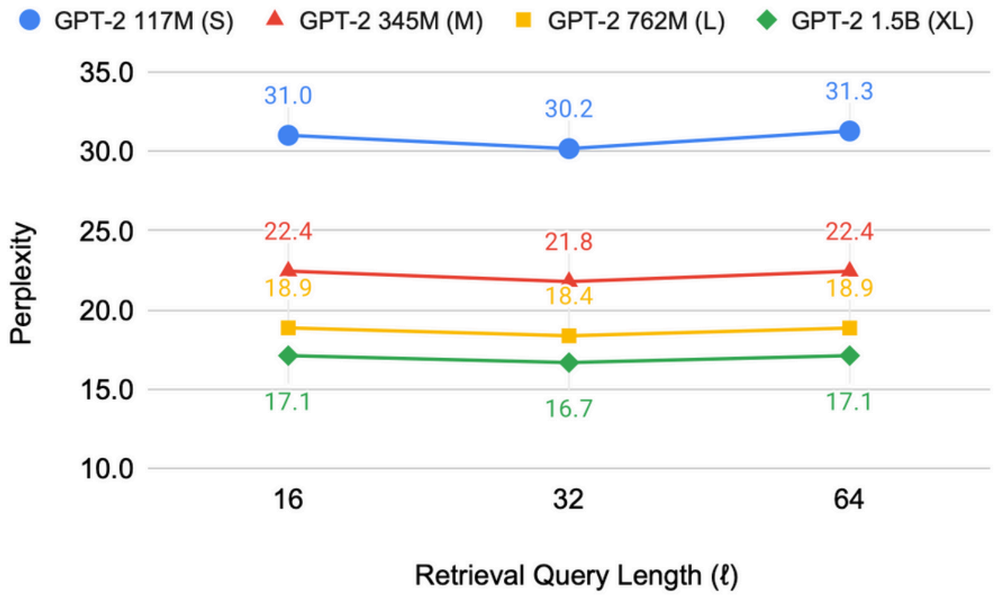


Figure 6: An analysis of perplexity as a function of *the number of tokens in the query* ℓ for BM25 on the development set of WikiText-103. In the appendix, we show similar trade-offs for dense retrievers within WikiText-103. Throughout the paper, we use a query length of $\ell = 32$ tokens.

BM25 Significantly Outperforms Dense Retrievers

- Evaluated on WikiText-103 with GPT-2 Small and GPT-2 XL
- Sparse BM25 achieves the lowest perplexity among all retrievers
- Dense retrievers (BERT, Contriever, Spider) underperform in zero-shot LM tasks
- Shows that simple lexical retrieval is surprisingly effective

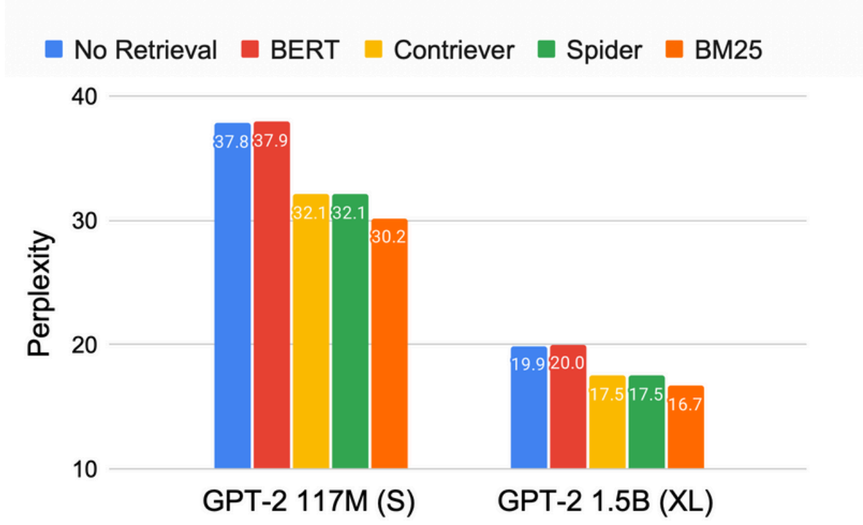


Figure 3: The performance of four off-the-shelf retrievers used for In-Context RALM on the development set of WikiText-103. All RALMs are run with $s = 4$ (*i.e.*, retrieval is applied every four tokens). For each RALM, we report the result of the best query length ℓ (see Figures 6, 9, 10).

Retrieval and Reranking Improve LM Performance

- In-Context RALM significantly reduces perplexity across all datasets and model sizes.
- BM25 retrieval alone achieves improvements equivalent to 2–3× model scaling.
- Zero-shot reranking provides additional gains without training the LM.
- Predictive reranking yields the best overall performance, reinforcing the value of LM-aware document selection.
- Improvements generalize across Wikipedia, scientific text, legal text, Q&A, and news domains.

Model	Retrieval	Reranking	WikiText-103	RealNews	ArXiv	Stack Exch.	FreeLaw
			word ppl	token ppl	token ppl	token ppl	token ppl
GPT-2 S	–	–	37.5	21.3	12.0	12.8	13.0
	BM25 §5	–	29.6	16.1	10.9	11.3	9.6
	BM25	Zero-shot §6.1	28.6	15.5	10.1	10.6	8.8
	BM25	Predictive §6.2	26.8	–	–	–	–
GPT-2 M	–	–	26.3	15.7	9.3	8.8	9.6
	BM25 §5	–	21.5	12.4	8.6	8.1	7.4
	BM25	Zero-shot §6.1	20.8	12.0	8.0	7.7	6.9
	BM25	Predictive §6.2	19.7	–	–	–	–
GPT-2 L	–	–	22.0	13.6	8.4	8.5	8.7
	BM25 §5	–	18.1	10.9	7.8	7.8	6.8
	BM25	Zero-shot §6.1	17.6	10.6	7.3	7.4	6.4
	BM25	Predictive §6.2	16.6	–	–	–	–
GPT-2 XL	–	–	20.0	12.4	7.8	8.0	8.0
	BM25 §5	–	16.6	10.1	7.2	7.4	6.4
	BM25	Zero-shot §6.1	16.1	9.8	6.8	7.1	6.0
	BM25	Predictive §6.2	15.4	–	–	–	–

Table 1: Perplexity on the test set of WikiText-103, RealNews and three datasets from the Pile. For each LM, we report: (a) its performance without retrieval, (b) its performance when fed the top-scored passage by BM25 (§5), and (c) its performance when applied on the top-scored passage of each of our two suggested rerankers (§6). All models share the same vocabulary, thus token-level perplexity (*token ppl*) numbers are comparable. For WikiText we follow prior work and report word-level perplexity (*word ppl*).

Motivation for Reranking

- BM25's top result is not always the best document for LM conditioning.
- Oracle selection among the top-16 retrieved documents shows much better perplexity.
- Indicates high untapped potential in document selection.
- Reranking aims to approximate the oracle without LM retraining.

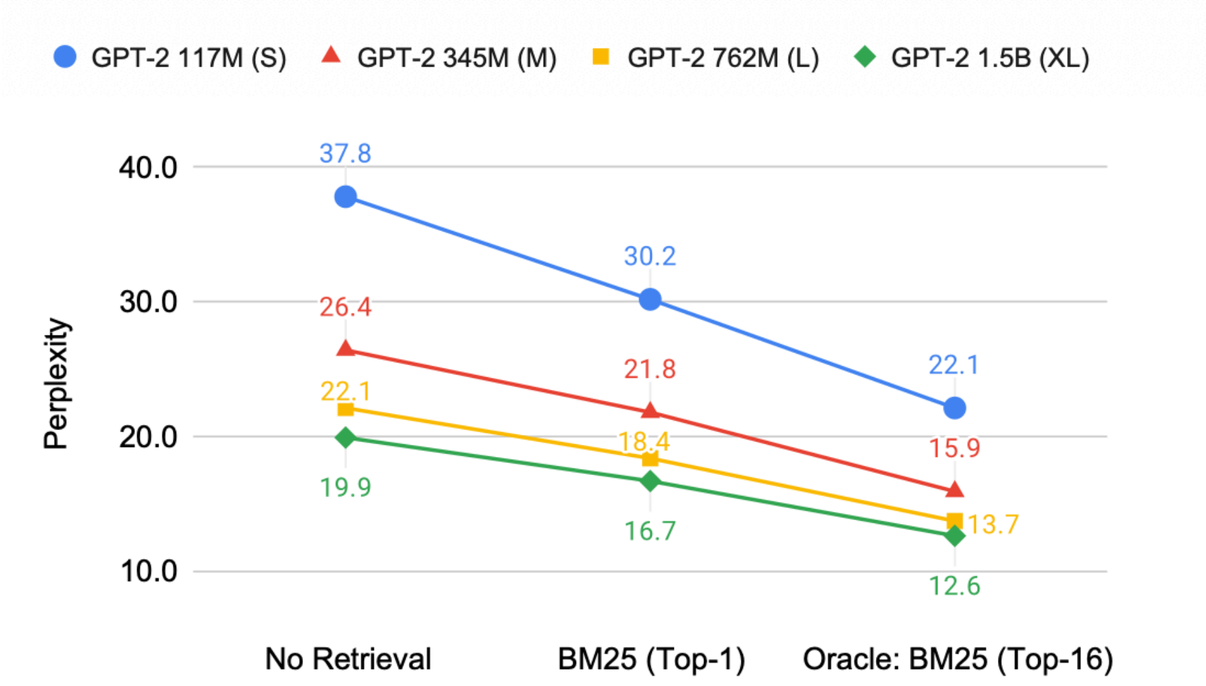


Figure 7: *Potential for gains* from reranking: perplexity improvement (on the development set of WikiText-103) from an oracle that takes the best of the top-16 documents retrieved by BM25 rather than the first.

Zero-Shot Reranking

- LM reranks the top-k BM25 documents by scoring their compatibility with the prefix.
- No training required; works even with small LMs.
- Consistently improves perplexity across GPT-2 sizes.
- Enables reranking in scenarios where main LM log-probs are unavailable (e.g., API-only models).

Model	Reranking Model	WikiText-103 word ppl	RealNews token ppl
GPT-2 345M (M)	GPT-2 110M (S)	20.8	12.1
	GPT-2 345M (M)	20.8	12.0
GPT-2 762M (L)	GPT-2 110M (S)	17.7	10.7
	GPT-2 762M (L)	17.6	10.6
GPT-2 1.5B (XL)	GPT-2 110M (S)	16.2	9.9
	GPT-2 1.5B (XL)	16.1	9.8

Table 3: Perplexity for zero-shot reranking (§6.1) where the reranking models is smaller than the LM, or the LM itself. Reranking is performed on the top 16 documents retrieved by BM25. Using a GPT-2 110M (S) instead of a larger language model as a reranker leads to only a minor degradation.

Predictive Reranking

- A RoBERTa-based reranker is trained using LM probabilities as self-supervision.
- Learns to pick the document that best predicts upcoming tokens.
- Requires no LM training — only reranker is updated.
- Consistently outperforms BM25 and zero-shot reranking across GPT-2 models.
- Closes much of the gap toward oracle retrieval performance.

Model	Retrieval	Reranking	WikiText-103	RealNews	ArXiv	Stack Exch.	FreeLaw
			word ppl	token ppl	token ppl	token ppl	token ppl
GPT-2 S	–	–	37.5	21.3	12.0	12.8	13.0
	BM25 §5	–	29.6	16.1	10.9	11.3	9.6
	BM25	Zero-shot §6.1	28.6	15.5	10.1	10.6	8.8
	BM25	Predictive §6.2	26.8	–	–	–	–
GPT-2 M	–	–	26.3	15.7	9.3	8.8	9.6
	BM25 §5	–	21.5	12.4	8.6	8.1	7.4
	BM25	Zero-shot §6.1	20.8	12.0	8.0	7.7	6.9
	BM25	Predictive §6.2	19.7	–	–	–	–
GPT-2 L	–	–	22.0	13.6	8.4	8.5	8.7
	BM25 §5	–	18.1	10.9	7.8	7.8	6.8
	BM25	Zero-shot §6.1	17.6	10.6	7.3	7.4	6.4
	BM25	Predictive §6.2	16.6	–	–	–	–
GPT-2 XL	–	–	20.0	12.4	7.8	8.0	8.0
	BM25 §5	–	16.6	10.1	7.2	7.4	6.4
	BM25	Zero-shot §6.1	16.1	9.8	6.8	7.1	6.0
	BM25	Predictive §6.2	15.4	–	–	–	–

Table 1: Perplexity on the test set of WikiText-103, RealNews and three datasets from the Pile. For each LM, we report: (a) its performance without retrieval, (b) its performance when fed the top-scored passage by BM25 (§5), and (c) its performance when applied on the top-scored passage of each of our two suggested rerankers (§6). All models share the same vocabulary, thus token-level perplexity (*token ppl*) numbers are comparable. For WikiText we follow prior work and report word-level perplexity (*word ppl*).

In-Context RALM for Open-Domain QA

- Evaluated on NQ and TriviaQA using frozen LLaMA models.
- Open-book prompting with retrieved documents greatly improves accuracy.
- LLaMA-13B: 12% → 31% (NQ), 54.8% → 60.1% (TriviaQA).
- Most gains achieved with only 1–2 documents.
- Demonstrates strong generalization of In-Context RALM beyond language modeling.

Model	Retrieval	NQ	TriviaQA
LLaMA-7B	-	10.3	47.5
	DPR	28.0	56.0
LLaMA-13B	-	12.0	54.8
	DPR	31.0	60.1
LLaMA-33B	-	13.7	58.3
	DPR	32.3	62.7

Table 4: Zero-shot results of In-Context RALM on the test set of Natural Questions and TriviaQA measured by exact match. In the open-book setting, we include the top two documents returned by DPR.

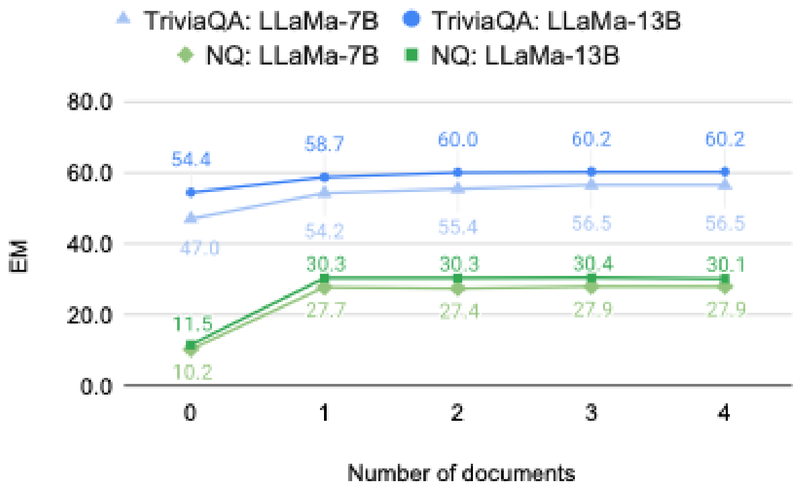
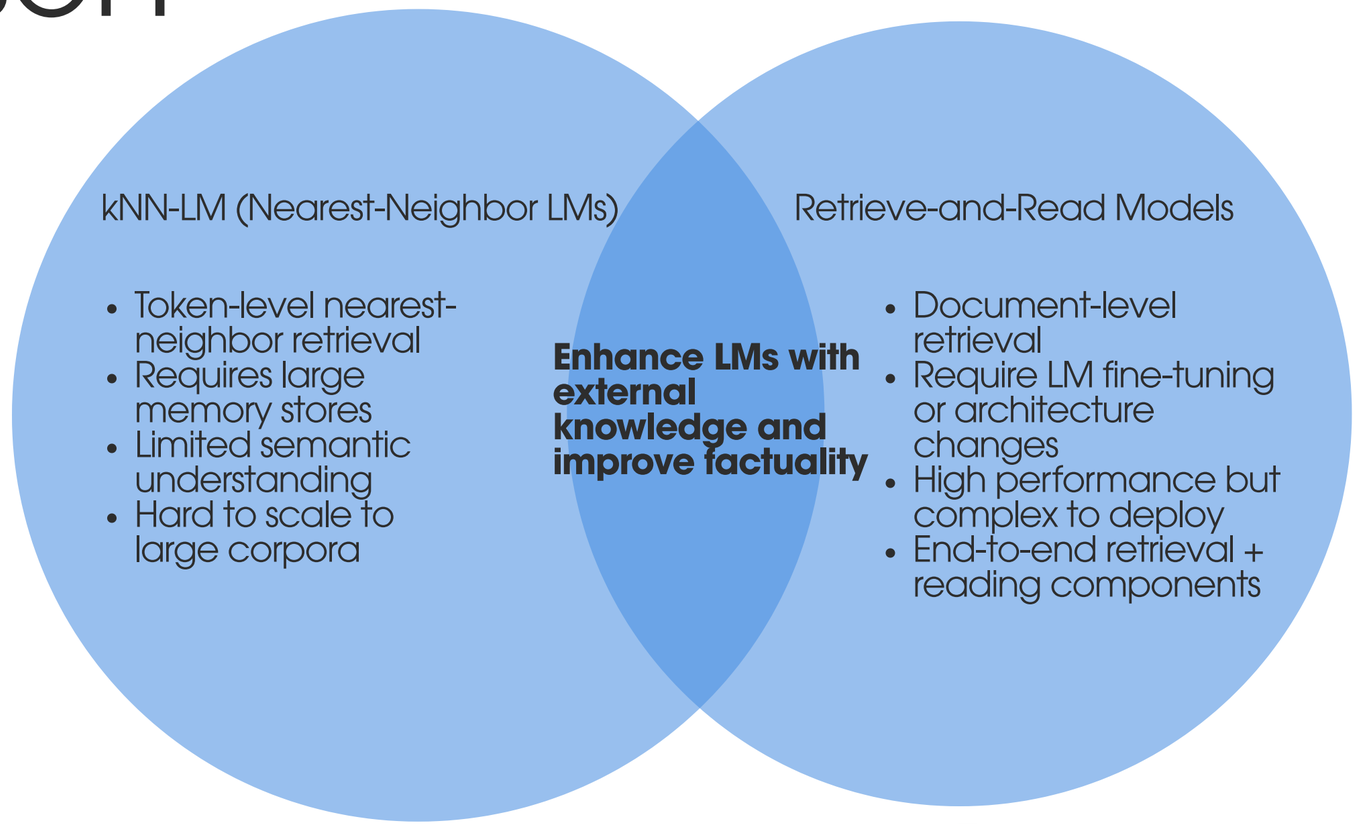


Figure 8: Zero-shot performance of In-Context RALM on the development set of Natural Questions and TriviaQA, when varying the number of documents (retrieved by DPR) shown in-context.

Comparison



In-Context RALM (This Work)

- Simple document prepending
- No LM modification or training required
- Works with frozen or API-only models
- Achieves gains comparable to much larger LMs

Any Questions ?